AI Usage Log - Kaila

CareerNet, database backend. AI tools: Claude and ChatGPT. This log covers the technical questions I worked through with AI and how I checked the answers.

Sprint 1 - Windows Setup and User Roles

Branches: fix/uvloop-windows and add-role-column

AI Tool: Claude

Task 1 - Python Virtual Environment on Windows

Prompt: "I'm setting up the project on Windows and python3 isn't being recognized, even though Python is installed. Why would that happen, and what's the right way to create and activate a virtual environment here?"

Purpose: Environment setup and troubleshooting

How AI helped:

Explained that the Microsoft Store Python alias can intercept the python3 command on Windows.

Suggested using the py launcher to create the virtual environment.

Showed how to activate it in PowerShell and how to handle the execution policy restriction.

Decision: Accepted

Reflection: I learned that Python commands can differ between operating systems and that virtual environments keep project dependencies separate. I also got more comfortable troubleshooting setup issues instead of immediately reinstalling software.

Task 2 - uvloop Failing to Install

Prompt: "Installing the requirements fails on uvloop for me but works for teammates on Mac. Is uvloop just not supported on Windows? And how can I fix it without breaking it for everyone else?"

Purpose: Dependency management

How AI helped:

Confirmed that uvloop does not support Windows.

Explained that Uvicorn falls back to the default event loop without it.

Suggested a platform marker in requirements.txt so Windows skips uvloop while macOS and Linux still install it.

Decision: Modified to match the project's cross-platform requirements

Reflection: Some installation errors come from the package itself rather than my setup. A platform marker let me fix it for Windows without changing anything for teammates on other systems.

Task 3 - Adding a User Role Column

Prompt: "The app needs to tell job seekers and recruiters apart. What's a good way to add a role to our existing SQLAlchemy User model so it only accepts those two values and lines up with what the frontend already uses?"

Purpose: Database design

How AI helped:

Reviewed the existing User model.

Recommended an enum with the values JOB_SEEKER and RECRUITER.

Pointed out that the values must match the ones the frontend uses.

Suggested Job Seeker as the default so existing users stay valid.

Decision: Modified to match the existing frontend values

Reflection: Enums are useful when a column should only accept specific values. I also saw how a database change has to stay consistent with what other teammates have already built.

Task 4 - Testing the Role Column

Prompt: "How can I actually test the new role field against a database before opening a pull request, without risking the shared one or exposing credentials?"

Purpose: Testing approach

How AI helped:

Suggested a local PostgreSQL database for testing.

Recommended keeping credentials in a .env file that Git ignores.

Explained the difference between reviewing a model change and running it against a real database.

Decision: Accepted, with the testing limitations noted

Reflection: Code review and testing against a database are separate checks, and both matter. Keeping credentials out of the code is part of doing database work safely.

Sprint 2 - KA-1: Profile, Resume File and Job Tables

Branch: feature/KA-1-profile-resume-job-tables

AI Tool: ChatGPT

Task 5 - Understanding Alembic

Prompt: "I keep seeing Alembic in our setup, but I don't really get what it adds if SQLAlchemy already defines the tables. What's the relationship between the two, and why do migrations matter for a team sharing one database?"

Purpose: Learning migration concepts

How AI helped:

Explained that SQLAlchemy describes the tables while Alembic records and applies changes to them.

Introduced revisions and how Alembic tracks which version a database is at.

Explained why a shared database needs every change tracked.

Decision: Accepted

Reflection: Before this I didn't understand why Alembic was needed. Now I see the models as the target and migrations as the history of how the database got there, which is what lets every teammate end up with the same schema.

Task 6 - Resume Access: Database or Backend?

Prompt: "Recruiters should see applicants' resumes, but job seekers should only see their own. Is that something the database should enforce, or is it the backend's job? I don't want to over-build the tables."

Purpose: Database design

How AI helped:

Explained that a foreign key links a resume to its owner but doesn't decide who can view it.

Separated storing relationships (the database) from access rules (backend authorization).

Helped me decide whether the tables needed any extra columns for this.

Decision: Kept the original database design

Reflection: Storing data and controlling access to it are separate responsibilities. The permission rules belong in the backend, outside my task, so I kept the tables simple.

Task 7 - Designing the Three Models

Prompt: "I need Profile, ResumeFile and Job models that fit our existing structure. What types and constraints make sense? For example, how should a resume file be stored, and should a job always need a recruiter?"

Purpose: Model implementation

How AI helped:

Walked through primary keys, foreign keys and the unique rule on the profile's user id.

Explained why resume files use LargeBinary.

Explained why recruiter_id has to be nullable, since sample jobs have no recruiter.

Helped choose data types for each column.

Decision: Modified to match the existing project structure

Reflection: I got a clearer picture of how models map to real tables and how constraints keep data consistent. I also got more comfortable building inside the existing structure instead of starting something separate.

Task 8 - Model Import Error

Prompt: "Python says it can't import my ResumeFile model, but the file is right there. What usually causes this?"

Purpose: Debugging

How AI helped:

Spotted that the import didn't match the filename resumeFile.py.

Explained how Python resolves module names, including capitalization.

Suggested a quick import test to confirm the fix.

Decision: Modified to keep my chosen filename

Reflection: A small naming mismatch is enough to break an import. Knowing how Python finds modules makes these errors much faster to track down.

Task 9 - Connecting Alembic to Our Models and Database

Prompt: "How does Alembic actually find our models and connect to Postgres? I want it to use the same database URL as the app instead of a hard-coded one."

Purpose: Configuration

How AI helped:

Explained the roles of alembic.ini and env.py.

Showed how importing the models in App/models/__init__.py lets Base.metadata see them.

Helped point Alembic at the existing environment-based connection.

Decision: Modified to match the existing project configuration

Reflection: If a model isn't imported, Alembic can't see it and the migration comes out empty. I also understood why the database URL should come from environment variables rather than the code.

Task 10 - Tables Exist, but No Migration History

Prompt: "Our database already has all the tables, yet Alembic says there's no migration history and no differences. How can both be true, and how do I start tracking without touching the existing tables?"

Purpose: Database analysis

How AI helped:

Explained that tables existing and Alembic tracking them are two separate things.

Clarified why the schema check found no differences.

Explained the role of an initial migration as a baseline.

Decision: Accepted

Reflection: Having the right tables doesn't mean changes are being tracked. A migration history is what makes the schema reproducible, and a database shared by the whole team needs extra care.

Task 11 - Writing the Initial Migration

Prompt: "Can you help me write an initial migration that matches our existing schema exactly? I want to compare it against the real tables rather than just trusting what gets generated."

Purpose: Migration implementation

How AI helped:

Explained the structure of a revision file, including the upgrade and downgrade functions.

Helped draft the migration for the existing tables.

Guided checking columns, foreign keys, unique constraints and indexes against PostgreSQL.

Helped find and remove a redundant unique constraint on email.

Decision: Modified to match the existing database schema

Reflection: Comparing the generated migration with the real tables caught a duplicate constraint. Generated code needs to be verified, not assumed correct.

Task 12 - Alembic Dialect-Loading Error

Prompt: "Alembic is throwing a dialect-loading error. Is that coming from my models or from the configuration? And can I preview the SQL without running anything on the shared database?"

Purpose: Configuration debugging

How AI helped:

Found a placeholder connection URL in the Alembic configuration.

Fixed the offline migration connection.

Explained the difference between generating SQL and running it, and helped produce an SQL preview.

Decision: Accepted

Reflection: The error came from configuration rather than the models. The SQL preview gave me a way to check a migration without changing the shared database.

Sprint 2 - KA-1 Wrap-up and SA-4: Applications Table

Branch: feature/SA-4-applications-table (Issue #89)

AI Tool: Claude

Task 13 - Applying the KA-1 Migration to the Shared Database

Prompt: "KA-1 is merged, but alembic current prints nothing on Neon. My migration is an initial schema that also creates the users table, which already exists. If I run upgrade head, won't it fail? What's the safest way to get Alembic in sync without touching tables the whole team is using?"

Purpose: Migration strategy on a shared database

How AI helped:

Confirmed that running the migration would fail because it tries to create the users table again.

Suggested checking which tables already existed before deciding anything.

Explained alembic stamp head, which records the migration as applied without running it.

Pointed out that the migration file should stay as it is, so someone setting up a fresh database can still run it normally.

Decision: Accepted, after confirming in pgAdmin that all four tables already existed

Reflection: I learned the difference between applying a migration and recording it. Stamping was the right choice here because the tables were already there; if any had been missing, the fix would have been different. Checking the real state of the database first is what made this a safe decision instead of a guess.

Task 14 - Designing the Applications Table

Prompt: "For SA-4, an application connects a user, a job and the resume they used. How should the foreign keys behave when one of those gets deleted? And what's the cleanest way to stop someone from applying to the same job twice?"

Purpose: Database design

How AI helped:

Suggested a unique constraint on the user and job together, so the database itself blocks duplicate applications.

Explained the delete options: CASCADE removes applications when a user or job is deleted, while SET NULL keeps an application if its resume is deleted.

Pointed out that without SET NULL, deleting a resume used in an application would fail, which would affect the KA-3 delete function.

Decision: Modified. I kept my own version of the model, which uses database-side defaults, rather than the restyled version AI suggested.

Reflection: I hadn't thought about how one table's delete rules affect another feature. Thinking through the delete behaviour made me picture what happens to real data when users or jobs are removed. A constraint in the database is also stronger than a check in the app code, because it can't be skipped.

Task 15 - ImportError on the New Model

Prompt: "Alembic fails with 'cannot import name Application', but the file and class are there. What am I missing?"

Purpose: Debugging

How AI helped:

Asked to see the actual file contents instead of guessing.

Found a typo in the class name (Applicaiton).

Decision: Accepted

Reflection: Looking at what is really in the file is better than assuming what should be there. This was the same kind of naming error as Task 8, so I recognized it faster.

Task 16 - Keeping the Model and Migration in Sync

Prompt: "The migration generated from my model looks right, but I want to make sure it only does what I expect. What should I check before applying it? And after pulling a teammate's change to the user model, how do I know whether it needs its own migration?"

Purpose: Migration review

How AI helped:

Listed what to check in the file: it builds on the KA-1 migration, only creates the applications table, includes the unique rule, and its downgrade only drops that table.

Explained that the model and its migration must match exactly, or the next generated migration picks up unexpected changes.

Suggested running alembic check after pulling, to confirm the teammate's change needed no migration.

Decision: Accepted

Reflection: A generated migration is a draft, not a guarantee. Reading the migration before applying it, and running alembic check after pulling, are habits I'll keep for every change to a shared database.

Sprint 2 - KA-2 to KA-4: Sample Jobs, Resume Storage and Job Search

Branches: feature/KA-2-seed-jobs (Issue #55), feature/KA-3-resume-functions (Issue #56) and feature/KA-4-job-search-query

AI Tool: Claude

Task 17 - A Seed Script That's Safe to Rerun

Prompt: "I'm writing a script to fill the jobs table with sample data the team can test search against. Since everyone shares the same database, how do I make sure running it twice doesn't double everything? And what makes sample data actually useful for testing?"

Purpose: Script design and test data

How AI helped:

Suggested checking for an existing job with the same title and company before adding it, so reruns skip duplicates.

Laid out the data so every category and location has the same number of jobs, several titles contain "Developer", and one job is closed.

Used our real Job columns and the project's existing database session, after I shared them.

Rolled back on errors, so a failed run doesn't leave half the data behind.

Decision: Accepted, after checking the data against the guide's requirements

Reflection: Making a script safe to rerun matters much more on a shared database. The sample data was also designed around the cases KA-4's search needs to handle, like keyword matches, filters and a closed job, so it's built for testing rather than just filling rows.

Task 18 - Resume Functions That Keep Users' Files Private

Prompt: "I need save, list, get, replace and delete functions for resumes that Lamees can call from her endpoints. How do I make sure a user can never reach someone else's resume? And since the files are stored in the table, how do I keep listing resumes from loading megabytes of data?"

Purpose: Data access design

How AI helped:

Had every function take the owner's user id and filter by it, so another user's resume returns nothing, as if it didn't exist.

Used SQLAlchemy's defer option so listing resumes skips the file data column.

Committed after every save, replace and delete, to avoid changes that seem saved but disappear.

Put the 5 MB limit and the allowed file types in a shared rules file for the backend and frontend.

Decision: Accepted

Reflection: Building ownership into every query means the endpoints don't have to remember to check it. I also learned that a large column like file data can be left out of a query, which matters once real resumes are stored.

Task 19 - Testing Data Isolation Between Users

Prompt: "How can I properly test these functions before handing them off? I especially want to prove one user can't touch another's resume, without leaving test data in the shared database."

Purpose: Testing

How AI helped:

Wrote a throwaway script that uses two real users and runs every function as both of them.

Checked that every attempt by one user to read, replace or delete the other's resume was refused.

Deleted the test resume at the end, so nothing was left behind.

Decision: Accepted. All 12 checks passed, and I kept the test script out of the repository.

Reflection: Testing the failure cases, not just the normal path, is what actually proves the privacy rule works. Cleaning up afterwards is part of testing responsibly on a shared database.

Task 20 - Building a Flexible Job Search

Prompt: "For KA-4, the search should take a keyword, location and category, but any of them can be missing. How do I build the query so missing filters are just skipped? And how do I return one page of results while still knowing how many matches there are in total?"

Purpose: Query design

How AI helped:

Built the query step by step, adding each filter only when a value is given.

Used a case-insensitive search across the title, company and description for the keyword, and case-insensitive exact matches for location and category.

Counted the total before limiting the results to one page, so the total isn't capped at the page size.

Hid closed jobs by default, with an option to include them later for recruiter pages.

Decision: Accepted. I asked for simple comments instead of docstrings, to match the style of our other files.

Reflection: Building the query in steps made the "skip missing filters" rule simple. The counting order was a subtle bug I wouldn't have noticed on my own, since a capped total still looks believable.

Task 21 - Testing Search Against Sample Data

Prompt: "How can I check the search actually behaves the way the guide describes, using the sample jobs, without changing anything in the database?"

Purpose: Testing

How AI helped:

Wrote a read-only throwaway script covering the guide's four cases: no filters, a keyword, location plus category, and no matches.

Added checks for paging, newest-first order, hidden closed jobs and job lookup.

Printed what each search returned, so I could check the results myself.

Decision: Accepted. All 16 checks passed, and I kept the test script out of the repository.

Reflection: Seeing the actual results helped me understand them. Searching "developer" also matched two designer jobs, because their descriptions mention working with developers. That's correct, since the keyword searches descriptions too. The KA-2 sample data made this much easier to test.

Final Outcome

Sprint 1: Fixed the uvloop installation issue on Windows with a platform marker, without affecting macOS or Linux.

Sprint 1: Added the user role column with the JOB_SEEKER and RECRUITER values, defaulting to Job Seeker.

Sprint 2 (KA-1): Built the Profile, ResumeFile and Job models and the initial migration. Merged, and the migration is stamped on Neon.

Sprint 2 (SA-4): Built the applications table and its migration, applied to Neon. Pull request open.

Sprint 2 (KA-2): Built the sample-jobs seed script. 18 jobs added, and rerunning adds none. Pull request open.

Sprint 2 (KA-3): Built the resume storage functions. 12 of 12 checks passed. Pull request open.

Sprint 2 (KA-4): Built the job search and lookup functions. 16 of 16 checks passed. Pull request open.

Overall Reflection: Over these two sprints, AI was most useful for understanding unfamiliar tools like Alembic, and for thinking through design decisions before writing code, such as how tables should behave when data is deleted and how to keep users' data private. The biggest thing I learned is to treat AI suggestions as a starting point and check them against reality. I inspected the real tables in pgAdmin before stamping a migration, read every generated migration before applying it, ran alembic check after pulling teammates' changes, and tested failure cases as well as the normal path. Working on a database the whole team shares made me much more careful about checking before changing anything.